
DOCUMENTO DE HANDOFF

Sistema de Gestión de Cirugías — *HospiTEC*

Curso: CE-1115 Seguridad de la Información
Semestre: I Semestre 2026
Profesor: MSc. Andrés Vargas Rivera
Institución: Instituto Tecnológico de Costa Rica
Versión: 1.0
Fecha: Abril 2026
Clasificación: **Confidencial**
Integrantes: Ricardo Borbón Mena
Jorge Guillén Campos
Felipe Jiménez Ulate
Carlos Rodríguez Segura
José María Vindas Ortiz

Stack tecnológico: Frontend React (Node.js) · Backend FastAPI (Python) · Base de datos PostgreSQL · Infraestructura Docker Compose

Índice

1. <i>Handoff</i>	2
1.1. Cómo ejecutar la aplicación	2
1.2. Dependencias	2
1.3. Variables de entorno	2
1.4. Qué incluye el entregable	3
1.5. Cómo usar la aplicación	3
1.6. Usuarios de prueba	4
1.7. Riesgos conocidos y limitaciones	4

1 Handoff

1.1 Cómo ejecutar la aplicación

Desde la raíz del repositorio, ejecutar:

```
docker compose up --build
```

Importante: El flag `--build` es obligatorio en el primer levantamiento para construir las imágenes. En ejecuciones posteriores puede omitirse si no hubo cambios en el código.

Una vez iniciados los contenedores, los puntos de acceso son:

Servicio	URL / Puerto	Descripción
Frontend	<code>http://localhost:3000</code>	Interfaz de usuario (React)
Backend (API)	<code>http://localhost:8000</code>	API REST (FastAPI)
Base de datos	<code>localhost:5432</code>	PostgreSQL

1.2 Dependencias

El único requisito de software para ejecutar el proyecto es:

- **Docker** con soporte para **Docker Compose v2** (`docker compose`, sin guion). Se puede verificar con:

```
docker compose version
```

No se requiere instalar Python, Node.js ni PostgreSQL de forma local, ya que todo el stack corre dentro de los contenedores.

1.3 Variables de entorno

El proyecto requiere un único archivo `.env` ubicado en el directorio raíz del proyecto, al mismo nivel que `docker-compose.yml`. A continuación se muestra la plantilla con los valores utilizados en el entorno de auditoría:

```
APP_NAME=Mi API
APP_VERSION=1.0.0
```

```
POSTGRES_DB=surgery_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
```

```
# Para activar 2FA real, reemplazar con credenciales propias de Resend
RESEND_API_KEY="re_XXXXXXXXXXXXXXXXXXXXXXXXX"
RESEND_API_EMAIL="sender@ejemplo.com"
ENABLE_2FA_EMAIL=false
```

```
DATABASE_URL=postgresql://postgres:postgres@db:5432/surgery_db
```

```
SECRET_KEY=una_clave_secreta_muy_larga_y_segura
ALLOWED_ORIGINS=["*"]
```

1.4 Qué incluye el entregable

El repositorio auditado se encuentra en:

<https://github.com/FelipeJU22/ProyectoSeguridad-DesarrolloYAuditoriaSeguridad>

El tag/release a auditar es **v1.0.0**. El repositorio contiene:

- **Código fuente:** Frontend en React (Node.js), Backend en FastAPI (Python) y configuración de Docker Compose.
- **Datos de prueba:** Script de *seed* SQL con usuarios de todos los roles precargados en la base de datos.
- **Políticas de seguridad:** Documento de políticas.

1.5 Cómo usar la aplicación

Todos los roles inician el flujo con **inicio de sesión** en `http://localhost:3000`. A continuación se describe el espacio de trabajo disponible para cada rol.

Paciente

- Al autenticarse, visualiza directamente el listado de sus cirugías.
- Puede **cancelar** una cirugía desde esa vista.
- Puede cambiar a la **vista de calendario** para ver sus cirugías organizadas por fecha.
- Puede cambiar a la **vista de documentos**, donde puede **subir**, **visualizar** y **eliminar** archivos PDF propios.

Cirujano

- Visualiza y **edita** las cirugías que le han sido asignadas.

- Dispone de la opción de **crear una nueva cirugía**, con un formulario que solicita todos los datos en una sola pantalla (tipo de cirugía, fecha, hora, paciente, anestesiólogo y asistente).
- Puede cambiar a la **vista de calendario**.

Asistente

- Visualiza las cirugías asignadas y puede **cambiar su estado**.
- Puede cambiar a la **vista de calendario**.

Anestesiólogo

- Visualiza (modo solo lectura) las cirugías que le han sido asignadas.
- Puede cambiar a la **vista de calendario**.

1.6 Usuarios de prueba

Los siguientes usuarios están precargados en la base de datos mediante el *seed* SQL incluido en el repositorio.

Datos sensibles — solo para el entorno de auditoría. Estos credenciales no deben usarse ni replicarse en ningún entorno productivo.

Rol	Correo	Contraseña
Cirujano	cirujano1@hospital.tec	Cirujano1234!
Cirujano	cirujano2@hospital.tec	Cirujano1234!
Anestesiólogo	anestesiologo1@hospital.tec	Anestesiologo1234!
Anestesiólogo	anestesiologo2@hospital.tec	Anestesiologo1234!
Asistente	asistente1@hospital.tec	Asistente1234!
Asistente	asistente2@hospital.tec	Asistente1234!
Paciente	paciente1@hospital.tec	Paciente1234!
Paciente	paciente2@hospital.tec	Paciente1234!
Paciente	paciente3@hospital.tec	Paciente1234!

1.7 Riesgos conocidos y limitaciones

1. **2FA con Resend API (limitación de configuración).** El segundo factor de autenticación requiere una cuenta activa en [Resend](#) con una API key válida y un correo de *sender* verificado. Si el auditor no dispone de estas credenciales, el sistema opera en

modo de prueba: el código 2FA aceptado será siempre `token1234`. Este comportamiento es un *fallback* exclusivo del entorno de auditoría y **no** constituye una vulnerabilidad de diseño intencional. Para activar el 2FA real se deben sustituir `RESEND_API_KEY` y `RESEND_API_EMAIL` en el `.env` y establecer `ENABLE_2FA_EMAIL=true`.

2. **Configuración de CORS dependiente del entorno.** Durante el desarrollo inicial se utilizó `ALLOWED_ORIGINS=["*"]` para facilitar la integración entre frontend y backend. Esta configuración fue posteriormente restringida a dominios específicos para mitigar riesgos de acceso no autorizado. Sin embargo, el servidor de desarrollo del frontend puede seguir mostrando configuraciones permisivas debido a sus propias limitaciones.
3. **Servidor de desarrollo del frontend sin headers de seguridad.** El frontend se ejecuta mediante el servidor de desarrollo de React (`npm start`) en `http://localhost:3000`. Este servidor no implementa headers de seguridad como *Content-Security-Policy (CSP)*, *X-Frame-Options* ni configuraciones restrictivas de CORS por diseño. Como resultado, herramientas de análisis como OWASP ZAP pueden reportar vulnerabilidades tales como:
 - Falta de CSP.
 - Configuración insegura de CORS.
 - Ausencia de protección contra *clickjacking*.

Estos hallazgos **no corresponden al backend ni a la lógica de la aplicación**, sino al entorno de desarrollo del frontend.

4. **Uso de directivas permisivas en CSP para entorno de desarrollo.** La política CSP implementada en el backend incluye directivas como `'unsafe-eval'` y `'unsafe-inline'` para permitir el correcto funcionamiento del entorno de desarrollo de React (hot reload, debugging). Estas configuraciones reducen el nivel de seguridad frente a ataques como XSS, pero son necesarias durante el desarrollo. En un entorno de producción, estas directivas deben eliminarse y reemplazarse por políticas más estrictas.
5. **Resultados de herramientas de auditoría condicionados por entorno.** Algunas vulnerabilidades detectadas por OWASP ZAP (como configuraciones de CSP o CORS) dependen del contexto en el que se ejecuta la aplicación. Dado que el entorno de evaluación utiliza contenedores Docker y un servidor de desarrollo para el frontend, ciertos hallazgos pueden no representar vulnerabilidades reales en un despliegue productivo, sino limitaciones del entorno de ejecución utilizado para la auditoría.